Welcome to our comprehensive guide on getting started with Phaser 4, a powerful and flexible open-source HTML5 game engine. In this course, you will dive deep into the world of Phaser 4, learning how to create engaging 2D games for the web. Whether you aim to build browser-based games, interactive prototypes, or mobile games deployable across various platforms, Phaser 4 offers the tools and functionalities you need.

Why Choose Phaser 4?

- **Cross-Platform Compatibility:** Build your game once and deploy it across multiple platforms. Phaser games run seamlessly in modern web browsers.
- **Open Source and Free:** Phaser is completely free to use and benefits from a vibrant and supportive community.
- **Versatility:** Ideal for a wide range of 2D games, from classic platforms and top-down adventures to puzzle games and more.
- **Modern JavaScript:** Built with modern JavaScript, making it efficient and enjoyable to work with.
- **Rich Feature Set:** Includes a robust rendering system, powerful physics engine, intuitive input management, advanced animation capabilities, audio management, and more.

Course Project: Frogger-Style Game

Throughout this course, you will build a Frogger-style game where your goal is to help a brave little character cross a dangerous road full of dragons. You will learn to dodge enemies and reach the treasure waiting on the other side. This project will provide hands-on experience with core gameplay mechanics such as movement, collisions, scene transitions, and more. By the end of the course, you will have a fully functional game that you can expand, customize, and showcase to others.

What You Will Learn

1. **Setting Up Your Phaser Project:** Covering the fundamental structure, methods for incorporating Phaser, and useful resources to accelerate your initial development.
2. **Implementing and Manipulating Game Objects:** Creating different game elements like sprites, images, and text, and tweaking their properties.
3. **Capturing Input:** Translating mouse clicks into movement for your main character.
4. **Game Object Interactions:** Detecting when game objects bump into each other, crucial for collecting items or dodging enemies.
5. **Engaging Effects:** Exploring effects like fade in and out and screen shake without complex techniques.
6. **Game Logic:** Defining when a player wins or loses and triggering game-ending scenarios.
7. **Scenes and Transitions:** Keeping your game organized and engaging by breaking it down into different scenes, like a menu and the main game, and smoothly transitioning between them.

Prerequisites

- Basic to intermediate knowledge of HTML, CSS, and JavaScript.
- A code editor (we recommend Visual Studio Code).
- A local web server to run your game files. We will cover how to set up a web server using the Live Server extension for VS Code early in the course. If you already have your own preferred web server setup, that will work too.
- Knowledge of how to bring in the Phaser library to get started.

About Zenva

Zenva is an online learning academy with over a million students. We feature a wide range of

courses for beginners and those looking to learn something new. Our courses are versatile, allowing you to learn in various ways, including video tutorials, lesson summaries, and following along with the instructor using included project files.

Are you ready to start building your own games? Let’s dive in and begin our Phaser 4 journey.

Welcome to your first step in game development with Phaser! In this lesson, we will guide you through setting up your project environment. A well-configured environment is crucial for a smooth development process. Let's get started.

Prerequisites

Before we begin, ensure you have the following:

- A code editor. We recommend **VS Code**, but feel free to use another if you prefer.
- A local web server. We will use the **Live Server** extension for VS Code.
- A copy of the project files. Please download and unzip them to an easily accessible location on your computer.

Why Use a Local Web Server?

A local web server provides a robust environment for developing interactive 2D games using Phaser. Here's why it's beneficial:

- Properly serves HTML, CSS, and JavaScript files, ensuring all assets and dependencies are loaded correctly.
- Correctly interprets relative file paths, preventing issues with resource loading.
- Automates the browser refresh process whenever you save changes to your code, providing instant feedback.

Installing Live Server

To streamline the development process, we will use the **Live Server** extension in VS Code. Follow these steps to install it:

1. Open VS Code and navigate to the extensions marketplace (usually represented by an icon with little squares).
2. In the search bar, type **Live Server**.
3. Look for the extension by **Ritwick Dey** and click **Install**.

Once installed, you will see a **Go Live** button in the bottom right-hand corner of VS Code.

Exploring the Project Files

Let's take a look at the starting project files:

- **phaser.js**: The Phaser library, essential for game development.
- **index.html**: The entry point for our game, containing the basic structure of an HTML document.
- **assets folder**: Contains all the images we will use for our game.

For our simple game, we will keep all JavaScript files inside the **source** folder and assets at the root of the **assets** folder.

Running the Project with Live Server

To see our game in the browser using Live Server, follow these steps:

1. Click the **Go Live** button in the bottom right corner of VS Code.
2. VS Code will open your default browser and serve your project through a local server (typically at **127.0.0.1:5500**).

3. You should see a blank page, which is expected since we have no content yet.

To test the hot reloading feature, add a paragraph tag with the text “Hello World” to the body section of **index.html** and save the file. Your webpage should update automatically.

Stopping Live Server

To stop Live Server, click the button in the bottom right-hand corner of VS Code. Alternatively, you can use the Command Palette:

1. Go to **View** in the toolbar.
2. Open the **Command Palette**.
3. Type **Live Server** and select the option to stop the server.

Recap

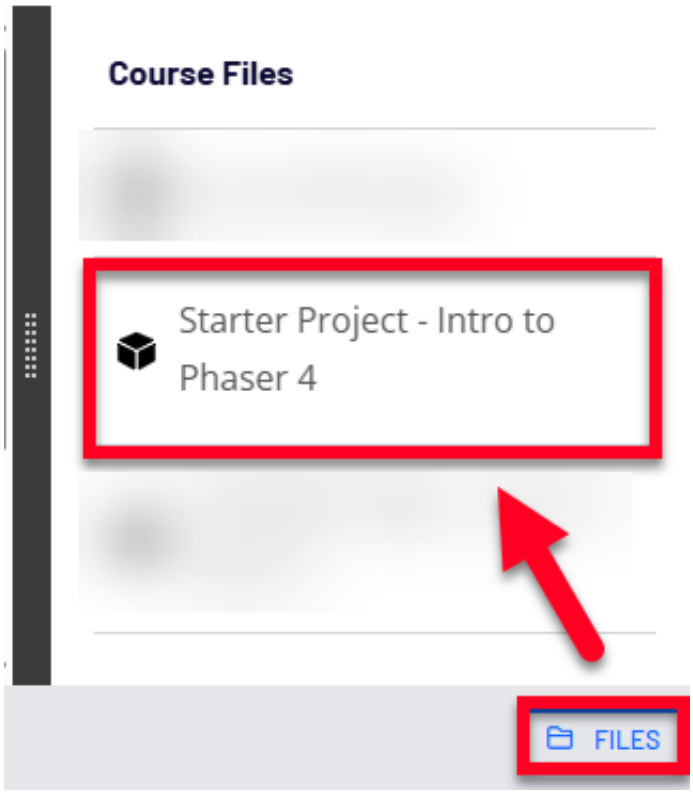
In this lesson, we:

- Installed the Live Server extension.
- Downloaded and unzipped our starter project files.
- Explored the project structure.
- Used Live Server to run our game in the browser.

Next, we will create a new Phaser Game instance and get something visible on our screen. Stay tuned for the exciting journey ahead!

For this course, we've included a Starter Project that already has a Phaser template set up for you. This project folder includes the Phaser 4 library and all the assets used in the course, so no extra download or setup steps are needed.

You can find this file in the **Files** tab in the bottom right-hand corner.



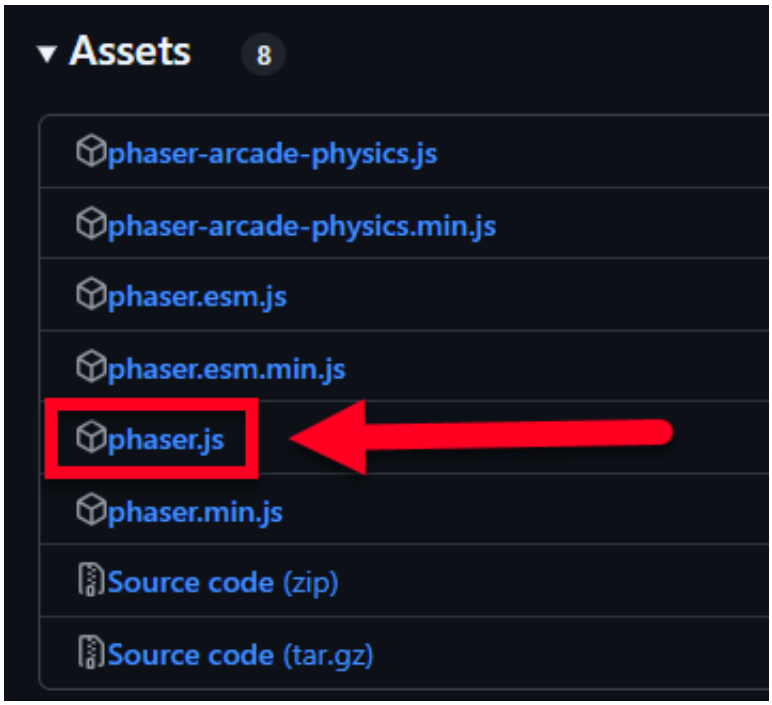
However, if you wish to download Phaser 4 and set up Phaser manually, you can follow the steps below!

Downloading Phaser

Our first step is to download the Phaser library itself. To do so:

1. Head to the official Phaser GitHub page here: https://github.com/phaserjs/phaser/releases/tag/v4.0.0-rc.4

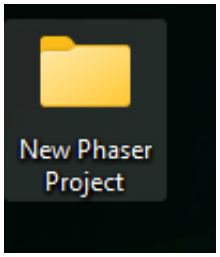
2. On this page, locate the "Assets" section and click the **phaser.js** file to download it.



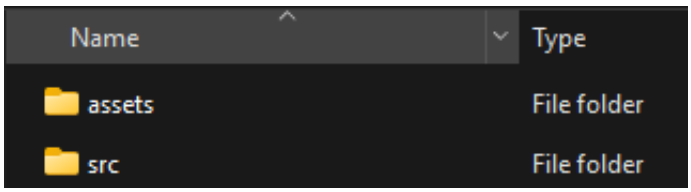
Setting up the Project

Now that you have the Phaser 4 library downloaded, you can set up your project. This can be done in any code editor.

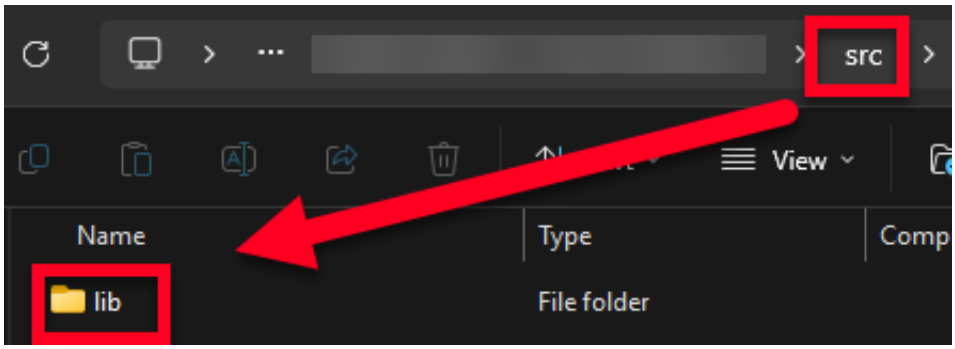
1. Create a new folder somewhere on your system to hold your project.



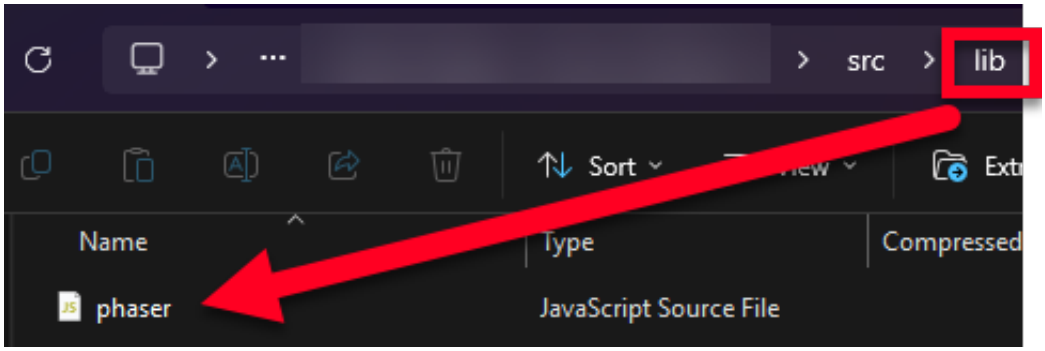
2. In this folder, create two new subfolders: one for “assets” and one called “src”.



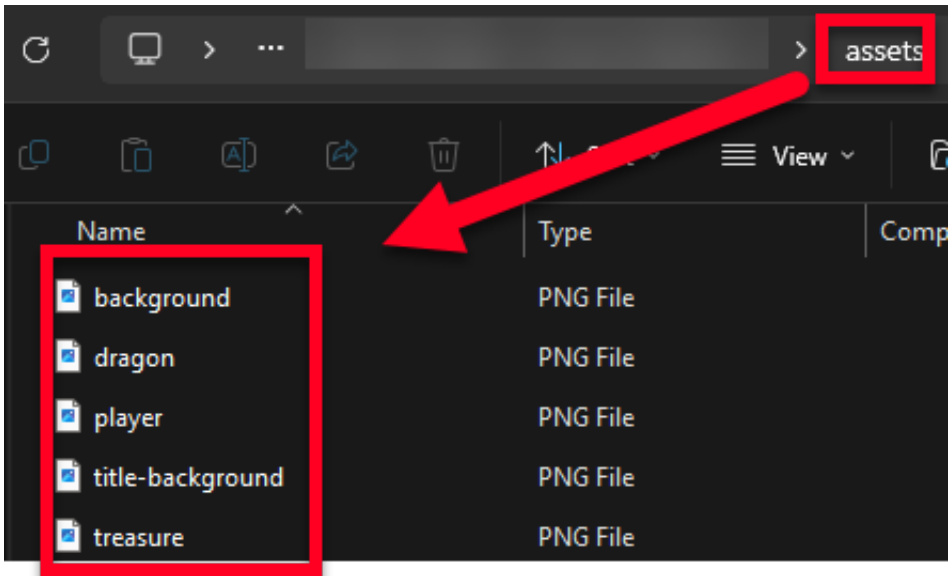
3. In the **src** folder, create a new folder called “lib”.



4. In the **lib** folder, add the **Phaser.js** file you downloaded from above.



5. In the assets folder, drag in any image assets your Phaser project will use. The assets for this course can be located in the Starter Project mentioned earlier.



6. In the parent project folder, create an **index.html** file.

7. In the **index** file, add the following code. This will set up a base webpage and pull in the Phaser library.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Learn Game Development at Zenva.com</title>
    <style>
      html, body {
        padding: 0px;
        margin: 0px;
      }
    </style>
  </head>
  <body>
    <script src="src/lib/phaser.js"></script>
  </body>
</html>
```

And that’s it! Phaser is now ready to go onto your machine (whether for this project or future projects)!

Welcome back! In this lesson, we will create our first Phaser game and launch it directly in the browser. The course provides a project template with Phaser 4 already included, allowing us to focus on coding without worrying about setup. Let’s dive right in!

Setting Up the Project

To begin, we need to create a new JavaScript file for our project. This file will serve as the main entry point for our web application.

1. Create a new file named main.js under the src folder.
2. Connect this file to our index.html page by adding a script tag in the body section:

Creating a Phaser Game Instance

Now, let’s create our Phaser game instance. This is the core of our Phaser project and will handle the game loop and rendering.

1. In main.js, create a new Phaser game instance:

```
const game = new Phaser.Game(config);
```

When you create an instance of the Phaser game class, Phaser sets up a canvas element and starts the game loop automatically.

Configuring the Game Instance

We can configure the game instance by passing an optional object when creating it. This object allows us to set properties like background color, canvas size, and scaling mode. To do so:

1. Create a configuration object:

```
const config = {
  // Configuration properties will go here
};
```

2. Set the background color of the canvas element:

```
const config = {
  backgroundColor: '#000000',
};
```

3. Set the size of the game canvas. You can use percentage values to make the canvas take up the full width and height of the webpage:

```
const config = {
  backgroundColor: '#000000',
```



```
  scale: {
    width: '100%',
    height: '100%',
  },
};
```

For our use case, we will use a hard-coded value for the game dimensions and scale it to fit:

```
const config = {
  backgroundColor: '#000000',
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
};
```

4. Set the rendering type:

```
const config = {
  backgroundColor: '#000000',
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
  type: Phaser.AUTO,
};
```

The type property tells Phaser which renderer to use. Phaser supports two renderers: Canvas and WebGL. By default, Phaser will use WebGL if supported; otherwise, it will fall back to Canvas.

Adding a Game Scene

Phaser uses scenes to divide the game into logical sections. A scene can be a level, a menu, or a screen in your game. Let’s create our first game scene.

1. Create a new folder named scenes under the src folder.
2. Inside the scenes folder, create a new file named game-scene.js.
3. Define a new Phaser scene class:

```
export class GameScene extends Phaser.Scene {
  constructor() {
    super({ key: 'GameScene' });
  }
}
```

4. Import the game scene class in main.js and pass it to the Phaser game configuration:

```
import { GameScene } from './scenes/game-scene.js';

const config = {
  backgroundColor: '#000000',
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
  type: Phaser.AUTO,
  scene: GameScene,
};

const game = new Phaser.Game(config);
```

That’s it! Our Phaser game is now up and running. There’s nothing visible yet, but the game is loaded, looping, and ready for action.

Challenges

Before we wrap up, here are a few challenges for you:

- Update your game configuration to have your game size be 1024×768.
- Update the default background color of your game to be a light blue.

In the next lesson, we will start adding visuals like background images, game characters, and more. Stay tuned!

In the previous video, we challenged you to update your game configuration to set the game size to 1024×768 pixels and change the background color to a light blue. In this lesson, we will walk through the solution to this challenge step-by-step.

Updating Game Size

To update the size of your game, you need to modify the width and height properties in the scale object of your Phaser game configuration. Here's how you can do it:

1. Open your main.js file.
2. Locate the configuration object where you set up your Phaser game.
3. Update the width and height properties within the scale object to 1024 and 768, respectively.

Here is the code snippet that demonstrates these changes:

```
// set the configuration of the game
const config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  scene: GameScene,
  scale: {
    width: 1024,
    height: 768,
    mode: Phaser.Scale.FIT,
  },
  backgroundColor: '#000000'
};

// create a new game, pass the configuration
const game = new Phaser.Game(config);
```

After saving these changes, open your browser and inspect the canvas element. You should see that the default width and height properties are applied based on your new settings. Additionally, if you resize the window, Phaser will automatically resize the canvas element to maintain the new aspect ratio.

Reverting to Original Size

For the purposes of this course, we will revert the game size back to 640×360 pixels. Update the width and height properties as follows:

```
// set the configuration of the game
const config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  scene: GameScene,
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
  backgroundColor: '#000000'
};
```



```
// create a new game, pass the configuration
const game = new Phaser.Game(config);
```

Updating Background Color

To update the background color of your game, you need to modify the backgroundColor property in your Phaser game configuration. For this challenge, we will set the background color to a light blue using the hexadecimal color code #028AF8.

Here is the updated code snippet:

```
// set the configuration of the game
const config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  scene: GameScene,
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
  backgroundColor: '#028AF8'
};

// create a new game, pass the configuration
const game = new Phaser.Game(config);
```

After saving these changes, refresh your browser. You should see that the canvas color has been updated to the specified light blue color.

Summary

In this lesson, we covered how to update the game size and background color in your Phaser game configuration. These changes are essential for customizing the appearance of your game. In the next lessons, we will continue to build on these foundations to create more complex and interactive game elements.

In this lesson, we will bring our game to life by adding images to the screen. Up until now, we have been working behind the scenes, but now it's time to see some real visuals show up in the game. The first step is to load our game assets. In our project folder, under the assets directory, we have four images that we will use for our background, player, enemy dragon, and the treasure we need to collect.

To render these images in our game, we need to tell Phaser, our game engine, that we want to load these assets. After informing Phaser about the images we want to load, we need to wait until our assets are available before we attempt to render them on the screen. We can do all this in our scene class by relying on our scene lifecycle.

Phaser Scene Lifecycle

In Phaser, our Phaser scene has four core main methods that Phaser invokes at particular points in our game once our scene is created. These methods are part of our core lifecycle for our scene:

- **init**: Called once our scene is initialized. It can be used for initiating certain parameters in our scene. This method is not always used in games, typically because we need to rely on our next method.
- **preload**: This is where all of our asset preloading takes place. We can tell Phaser about all of the assets, our images, audio files, or any other external files that we want to use in our game, and Phaser will queue those up and start loading them. Only once all of our files are loaded will Phaser then trigger our create method.
- **create**: Called once we can start creating our game objects and displaying them in our scene.
- **update**: Tied to our core game loop of Phaser. This method is called multiple times per second, or for each tick of our game loop. This method is really useful for when we need to check things all of the time in our game. We'll dive more into the update method later in our course.

To hook into the Phaser lifecycle for our scene, we just need to use those four methods in our class. Let's add those four methods now and log out a message to see what happens in our game.

Adding Lifecycle Methods

We will add the init, preload, create, and update methods to our scene class and log messages to the console to understand when each method is called.

```
// create a new scene
export class GameScene extends Phaser.Scene {
  constructor() {
    super({
      key: 'GameScene'
    });
  }

  // initialize game properties
  init() {
    console.log('init method called');
  }

  // load assets
  preload() {
    console.log('preload method called');
  }
```

```
  // called once after the preload ends
  create() {
    console.log('create method called');
  }

  // called for each step of our game loop
  update() {
    console.log('update method called');
  }
}
```

If we save and run our game, we should see our new console log statements being logged. Once Phaser creates an instance of our game scene and we start our scene, we'll see our init method is called, then our preload, then our create, and finally, we'll see that our update method is being called multiple times per second.

For the time being, I'm going to comment out our console log statement for the update method to reduce the noise in our console.

Loading Assets

Now we need to load in the assets that we want to use for our game. We'll do this in our preload method. That way, we can use that image data inside our create method.

To load in our assets for our game, we can use the load property, which references our loader plugin in Phaser. Our loader plugin has a variety of built-in methods to handle different file types that we want to load in.

To load in the images we want to use for our game, we're going to use the image method. The image method expects two arguments: the first is the key of the asset we want to load, and the second will be our URL or our relative path to load that file.

Let's load in our background image for our game. To load in our background image, let's use background for our key. Our key will be important later when we want to create our game object. When Phaser loads our image data, Phaser is going to cache that data internally by this key we provide here. When we go to create our game object, we'll be able to provide that same key, and then that's how Phaser will know to use that image data to draw that image to our screen.

Our second argument will be the URL or our relative path to the image we want to load. So this will be assets/background.png.

```
// load assets
preload() {
  console.log('preload method called');
  // load images
  this.load.image('background', 'assets/background.png');
}
```


If we save, we'll see there's no change over in our game. All we've done so far is we've told Phaser we want to load this image asset. Under the hood, Phaser is going to load this image and wait until that image is available before transitioning to our create method.

Inside our create method, this is where we can create our game object and render our image out to our screen. That concludes this lesson. In our next lesson, we will continue working on adding images to our game.

In this lesson, we will continue working on adding images to our game. Specifically, we will focus on rendering a background image and understanding how Phaser's coordinate system works to position our game objects accurately.

Adding the Background Image

To render our background image, we need to use Phaser's game object factory. This factory provides several utility methods for creating game objects quickly. For our background image, we will use the add.image method.

Here is how you can add the background image to your game:

```
create() {
   this.add.image(0, 0, 'background');
}
```

This method expects three arguments:

- The first two arguments are the x and y coordinates where you want to position the game object.
- The third argument is the key of the image data you want to show in your game. This key must match the key used when loading the image in the preload method.

Understanding Phaser's Coordinate System

Phaser uses a 2D coordinate system where:

- The x-axis goes from left to right.
- The y-axis goes from top to bottom.

The top-left corner of the game canvas is at coordinates (0, 0). When you provide coordinates to the add.image method, Phaser positions the game object based on these coordinates.

Positioning the Background Image

By default, Phaser uses the center of the image as the origin point for positioning. This means that when you load an image game object and place it in the game, Phaser creates an origin point right in the center of the image.

For example, if you set the x coordinate to 100, the image will move to the right, with the center of the image at the coordinate (100, 0). Similarly, setting the y coordinate to 100 will move the image down, with the center of the image at the coordinate (0, 100).

Centering the Background Image

To center the background image, we need to use the game scene's width and height properties. These properties can be accessed from the scale manager on the scene.

Here is how you can grab the game's width and height:

```
const gameW = this.scale.width;
const gameH = this.scale.height;
```

To find the center of the screen, divide these values by 2. Now, update the game object's position using the setPosition method:

```
const bg = this.add.image(0, 0, 'background');
bg.setPosition(gameW / 2, gameH / 2);
```

Adjusting the Origin Point

As an alternative to updating the game object's x and y values for positioning, you can also update the image's origin point. By default, the origin is in the center of the image, but you can change it to the top-left corner using the setOrigin method:

```
bg.setOrigin(0, 0);
```

This will set the origin to the top-left corner of the image. When you set the position to the center of the screen, Phaser will start drawing the image from this origin point.

Recap

In this lesson, we covered the following steps to load and render a background image in our game:

1. Load the image asset in the preload method.
2. Create the image game object using this.add.image and provide the key to the asset.
3. Use the setPosition method to place the game object exactly where we want it in the game.
4. Use the width and height properties from the scale manager to calculate the center of the game.
5. Understand how origins and coordinates affect the placement of the game object.

Challenge

Now it's your turn to try loading and displaying the player's image in the game. Once the player is visible, update the game object's position to be in the bottom-right corner of the screen. Use the origin and the scale manager to achieve this.

Good luck, and happy coding!

Welcome back! In our previous video, we challenged you to load and display your player image in the game and position it in the bottom right-hand corner of the screen. In this lesson, we will walk through a possible solution to this challenge step-by-step.

Loading the Player Image

To begin, we need to load the player image into our game. This is done in the preload method of our game scene. The preload method is where we load all the assets our game will need, such as images, sounds, and data files.

Here is how you can load the player image:

```
preload() {
  // Load the player image
  this.load.image('player', 'assets/player.png');
  ...
}
```

In this code:

- 'player' is a unique key that we will use to reference this image later.
- 'assets/player.png' is the path to the image file.

Creating the Player Game Object

Once the image is loaded, we need to create a game object to display it on the screen. This is done in the create method.

```
create() {
  // Create the player game object
  const gameW = this.scale.width;
  const gameH = this.scale.height;

  const player = this.add.image(0, 0, 'player');

  ...
}
```

In this code:

- this.add.image creates a new image game object.
- (0, 0) specifies the initial position of the image on the screen.
- 'player' is the key we used to load the image in the preload method.

Positioning the Player in the Bottom Right Corner

To position the player in the bottom right corner of the screen, we need to use the game’s width and height properties. Here is how you can do it:

```
create() {
  const gameW = this.scale.width;
```



```
  const gameH = this.scale.height;

  const player = this.add.image(0, 0, 'player');

  // Position the player in the bottom right corner
  player.setPosition(gameW, gameH);

  ...
}
```

However, you might notice that the player is not fully visible. This is because the origin of the image is at the top-left corner by default. To fix this, we need to set the origin to the bottom right corner:

```
create() {
  const gameW = this.scale.width;
  const gameH = this.scale.height;

  const player = this.add.image(0, 0, 'player');

  // Position the player in the bottom right corner
  player.setPosition(gameW, gameH);

  // Set the origin to the bottom right corner
  player.setOrigin(1, 1);

  ...
}
```

In this code:

- player.setPosition(this.scale.width, this.scale.height) positions the player at the bottom right corner of the screen.
- player.setOrigin(1, 1) sets the origin of the image to the bottom right corner, making the player fully visible.

That’s it! You have now loaded and displayed your player image in the game and positioned it in the bottom right corner of the screen. In the next lessons, we will continue to build on this foundation and add more features to our game.

In this lesson, we will explore how to manipulate game objects in Phaser to add more personality to our game. Specifically, we will learn how to scale, flip, and layer our game objects. These techniques will allow us to create more dynamic and visually appealing games.

Understanding Layering in Phaser

Phaser renders game objects in the order they are added to the scene. This means that the first object added will be at the bottom layer, and subsequent objects will be layered on top. For example, if we add a background image first and then a player image, the player will appear on top of the background.

Here is a simple example to illustrate this:

```
// Adding background image first
const bg = this.add.image(0, 0, 'background');

// Adding player image second
const player = this.add.image(gameW, gameH, 'player');
```

In this case, the player will be visible on top of the background because it was added later.

Controlling Depth

To have more control over the rendering order, we can use the depth property. The depth property allows us to specify the layer order explicitly. Game objects with a higher depth value will be rendered on top of those with a lower depth value.

Here is how you can set the depth property:

```
// Setting depth for the player
player.setDepth(2);
```

In this example, the player will be rendered on top of any other game objects with a depth value of 1 or lower. Thus, we can arrange our code as shown below. Even though the player is rendered first, because of our depth setting, they will still appear on top of the background.

```
create() {
   console.log('create method called');
   const gameW = this.scale.width;
   const gameH = this.scale.height;
   console.log(gameW, gameH);

   const player = this.add.image(0, 0, 'player');
   player.setPosition(gameW, gameH);
   player.setOrigin(1, 1);
   player.setDepth(2);

   // create bg image
   const bg = this.add.image(0, 0, 'background');

   // place image in the center of the screen
```



```
  bg.setPosition(gameW / 2, gameH / 2);

  // change origin to the top-left corner
  // bg.setOrigin(0, 0);
}
```

Scaling Game Objects

Scaling a game object means changing its width and height. We can scale an object along the x-axis, y-axis, or both. This allows us to resize images without modifying the original image file.

To scale a game object, we use the setScale method:

```
// Scaling the player object
player.setScale(2, 2);
```

This will double the width and height of the player object. Similarly, you can shrink an object by setting the scale to a value less than 1:

```
// Shrinking the player object
player.setScale(0.5, 0.5);
```

You can also scale only one dimension:

```
// Scaling only the width
player.setScale(0.5, 1);

// Scaling only the height
player.setScale(1, 2);
```

If you provide only one argument to the setScale method, Phaser will apply that value to both the x and y axes:

```
// Scaling both width and height uniformly
player.setScale(0.5);
```

Adding and Scaling Enemies

Let’s add some enemies to our game and scale them differently. First, we need to load the enemy image in the preload method:

```
// Loading the enemy image
this.load.image('enemy', 'assets/dragon.png');
```


Next, we create enemy game objects and scale them. Besides using set scale, we can also modify the scaleX and scaleY properties directly:

```
const enemy1 = this.add.image(250, 180, 'enemy');
enemy1.scaleX = 2;
enemy1.scaleY = 2;
```

Alternatively, we can use properties known as **display width** and **display height**. In the backend, Phaser is already changing this when we use either of the previous methods. This is one more way, however, that we can adjust an object's scale.

```
const enemy2 = this.add.image(450, 180, 'enemy');
enemy2.displayWidth = 300;
```

Flipping Game Objects

Flipping a game object allows us to mirror it horizontally or vertically. This can be useful for creating variations of the same object without needing additional images.

To flip a game object, we use the flipX and flipY properties:

```
// Flipping the first enemy horizontally
enemy1.flipX = true;

// Flipping the first enemy vertically
enemy1.flipY = true;
```

Recap

In this lesson, we covered several important concepts:

- **Scaling:** Changing the size of game objects using the setScale method or by adjusting the scaleX, scaleY, or displayWidth properties.
- **Flipping:** Mirroring game objects horizontally or vertically using the flipX and flipY properties.
- **Depth:** Controlling the rendering order of game objects using the depth property.

With these techniques, you now have full control over how images look and layer in your game. In the next lesson, we will make things a little more dynamic by rotating our images and updating their behavior in real time.

In this lesson, we will add dynamic rotation and continuous updates to our sprites in Phaser, making them more lively and interactive. We'll utilize Phaser's scene lifecycle, specifically the update method, to create motion and animations. Let's get started by organizing our code and then implementing these features step-by-step.

Cleaning Up the Game Scene

Before we dive into rotations and updates, let's clean up and organize our game scene code:

- Remove the init method.
- Remove any console.log statements from the preload and create methods.
- Move the player game object creation code after the background game object creation code.
- Remove the code that updates the scale and flip properties of the enemies, keeping only one enemy on the screen.
- Update the background image code to use the origin property and remove the code that centers the game object.
- Update the player's position to x: 70 and y: 180.
- Update the player's scaling using setScale.

```
// create a new scene
export class GameScene extends Phaser.Scene {
  constructor() {
    super({
      key: 'GameScene',
    });
  }

  // load assets
  preload() {
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
  }

  // called once after the preload ends
  create() {
    // create bg image
    const bg = this.add.image(0, 0, 'background');

    // change origin to the top-left corner
    bg.setOrigin(0, 0);

    const player = this.add.image(70, 180, 'player');
    player.setScale(0.5);

    const enemy1 = this.add.image(250, 180, 'enemy');
  }

  // called for each step of our game loop
  update() {
    // console.log('update method called');
    }
  }
}
```

Rotating Game Objects

In Phaser, you can rotate a game object using angles or radians. Both methods rotate around the game object's origin point, which is the center by default. Let's explore how to rotate our enemy game object using both degrees and radians.

Rotating Using Degrees

To rotate our enemy game object by 45 degrees, we use the angle property:

```
enemy1.angle = 45;
```

Alternatively, you can use the setAngle method:

```
enemy1.setAngle(45);
```

Rotating Using Radians

To rotate using radians, we use the rotation property. Since one full rotation is 2 * Math.PI radians, rotating by 45 degrees is equivalent to Math.PI / 4 radians:

```
enemy1.rotation = Math.PI / 4;
```

Similarly, you can use the setRotation method:

```
enemy1.setRotation(Math.PI / 4);
```

Changing the Origin Point

By default, the origin point is the center of the game object. You can change the origin point to create different rotation effects. For example, to set the origin to the top-left corner:

```
enemy1.setOrigin(0);
```

Continuous Rotation Using the Update Method

The update method in Phaser runs continuously in the background, allowing you to create animations and check for conditions frame by frame. To make our enemy game object spin continuously, we need to increment its angle in the update method.

First, store a reference to the enemy game object as a property of the class:

```
this.enemy1 = this.add.image(250, 180, 'enemy');
```

Then, in the update method, increment the angle:

```
update() {
  this.enemy1.angle += 1;
}
```

This will make the enemy game object spin continuously.

Challenges

To reinforce what you've learned, try the following challenges:

1. Rotate the player game object counterclockwise over time.
2. Scale the player game object over time until it reaches twice its original dimensions.

Hint: Store a reference to the player game object and update its properties in the update method.

Recap

In this lesson, you learned how to:

- Rotate game objects using degrees and radians.
- Change the origin point for different rotation effects.
- Use the update method to create continuous animations.

With these tools, you can now bring your scenes to life with dynamic rotations and real-time updates. In the next lesson, we'll finish up this section and then start integrating player and goal objects into the game, eventually introducing enemies as well. Stay creative, and see you in the next one!

In this lesson, we will address the challenges presented at the end of the previous video. These challenges involve updating our player game object to rotate counterclockwise over time and scaling it until it reaches twice its original size. To achieve these tasks, we need to modify the properties of our player game object within the update method of our game scene.

Storing a Reference to the Player Game Object

To update the properties of our player game object, we need to store a reference to it in our class. This allows us to access and modify the game object within the update method. Here's how you can do it:

```
this.player = this.add.image(70, 180, 'player');
this.player.setScale(0.5);
```

Rotating the Player Game Object

To make the player game object rotate counterclockwise, we need to decrement its angle property within the update method. This can be done using either angles or radians. For this example, we will use angles:

```
update() {
    ...
    this.player.angle -= 1;
}
```

Scaling the Player Game Object

To scale the player game object over time, we need to increment its scaleX and scaleY properties within the update method. We will increase these properties by a small value, such as 0.01:

```
update() {
    ...
    this.player.scaleX += 0.01;
    this.player.scaleY += 0.01;
}
```

Controlling the Maximum Size

To ensure that the player game object does not grow indefinitely, we need to add a condition to stop scaling once it reaches a certain size. We will use an if statement to check if the scaleX property is less than 2:

```
update() {
    ...
    if (this.player.scaleX < 2) {
        this.player.scaleX += 0.01;
        this.player.scaleY += 0.01;
    }
}
```

Complete Update Method

Here is the complete update method with both the rotation and scaling logic included:

```
update() {
    ...
    this.player.angle -= 1;
    if (this.player.scaleX < 2) {
        this.player.scaleX += 0.01;
        this.player.scaleY += 0.01;
    }
}
```

Summary

In this lesson, we learned how to:

- Store a reference to the player game object in our class.
- Rotate the player game object counterclockwise over time.
- Scale the player game object over time until it reaches twice its original size.
- Control the maximum size of the player game object using a conditional statement.

By following these steps, you should now be able to update the properties of your player game object within the update method to achieve the desired behavior. This knowledge will be crucial as you continue to develop more complex game mechanics in future lessons.

In this article, we will address a common question that arises when beginning to work with Phaser: Should you modify game object properties directly or use built-in methods? Let’s explore this topic in a clear and concise manner.

Understanding Property Modification in Phaser

In Phaser, game object properties such as position, scale, rotation, and visibility can typically be changed in two ways:

- **Direct Property Assignment:** Modifying properties directly on the game object.
- **Setter Methods:** Using built-in methods to update properties.

Direct Property Assignment

Direct property assignment involves changing the properties of a game object directly. For instance, to modify the scaleX property, you can set both the x and y values individually, or set both at once using the scale property.

```
// Setting x and y values individually
gameObject.scaleX = 2;
gameObject.scaleY = 2;

// Setting both at once
gameObject.scale = 2;
```

Using Setter Methods

Setter methods provide a more structured way to update properties. For the scale property, you can use the setScale method. This method allows you to provide one argument to set both x and y values, or provide them individually.

```
// Setting both x and y values to the same value
gameObject.setScale(2);

// Setting x and y values individually
gameObject.setScale(2, 2);
```

Benefits of Using Setter Methods

- **Readability:** Setter methods make your code easier to read and understand.
- **Efficiency:** They allow you to update multiple properties at once.
- **Chaining:** Setter methods often support chaining, enabling you to perform multiple actions in a single line of code.

For example, you can chain methods to scale, set alpha, and rotate a game object:

```
player.setScale(2).setAlpha(0.5).setRotation(45);
```


When to Use Direct Property Assignment

While setter methods are generally preferred, there are situations where direct property assignment makes sense:

- When you need to change only one value.
- When the change is simple and direct assignment is more straightforward.

For instance, to increment the x position of a player sprite by 10:

```
player.x += 10;
```

Best Practices

1. **Use Setter Methods:** As a general rule, use setter methods for clarity, safety, and convenience.
2. **Be Consistent:** Maintain a consistent style in your code for better readability and maintenance.

Example Comparison

Using setter methods to chain multiple actions:

```
player.setScale(2).setAlpha(0.5).setRotation(45);
```

Versus direct property assignment:

```
player.scale = 2;
player.alpha = 0.5;
player.rotation = 45;
```

In conclusion, while both direct property assignment and setter methods are valid, setter methods offer several advantages that make them the preferred choice in most scenarios. Use what makes your code cleaner and more maintainable. Happy coding!

In this lesson, we will bring our player to life and give them a purpose by setting up a goal to reach. By the end of this lesson, you will see our player moving across the screen, interacting with a treasure chest goal. This is a great way to get familiar with handling user input and collision detection in Phaser.

Cleaning Up the Game Scene

Before we add our code, let's do a little bit of cleanup in our game scene. We will start by removing all the code in our update and create methods related to the enemy. This will leave us with just our player and background image game objects.

```
// create a new scene
export class GameScene extends Phaser.Scene {
  constructor() {
    super({
      key: 'GameScene',
    });
  }

  // load assets
  preload() {
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
  }

  // called once after the preload ends
  create() {
    // create bg image
    const bg = this.add.image(0, 0, 'background');

    // change origin to the top-left corner
    bg.setOrigin(0, 0);

    // create the player
    this.player = this.add.image(40, this.scale.height / 2, 'player');
    // we are reducing the width and height by 50%
    this.player.setScale(0.5);
  }

  // this is called up to 60 times per second
  update() {

  }
```

Handling Player Movement

To make our player move right across the screen when we click or touch the screen, we will use Phaser's built-in input detection. We will listen for mouse clicks or touches to trigger code in our game.

Detecting Input

To listen for input in our game, we can reference our input plugin from our update method. Here is how you can do it:

```
update() {
  if (this.input.activePointer.isDown) {
    console.log('input detected');
  }
}
```

In this code:

- this.input references the input manager on our Phaser scene, which is responsible for handling all input-related events in our game.
- activePointer is a Phaser pointer game object that points to the most recently active pointer object in our game. This could be a mouse click or a touch event on a mobile device.
- isDown checks if the button is currently being pressed down.

Moving the Player

Now that we are listening for input, we need to update our player's X position to move our player game object across the screen. We will reference our player game object on our scene and increase its X property by some value.

```
update() {
  if (this.input.activePointer.isDown) {
    this.player.x += 3;
  }
}
```

To make the player's movement smoother, we will add a new property to our class to keep track of our player's speed. This will allow us to modify the speed value later if needed.

```
init() {
  this.playerSpeed = 3;
}

update() {
  if (this.input.activePointer.isDown) {
    this.player.x += this.playerSpeed;
  }
}
```

Adding the Treasure Chest Goal

To give our player a purpose, we will introduce a treasure chest goal. First, we need to load in our asset for the treasure chest.

```
preload() {
  this.load.image('treasure', 'assets/treasure.png');
```



```
}
```

Next, we need to create an instance of our game object in our level. We will do this in our create method after creating our player. We will also adjust the position of our player and goal using the scale width and height of our scale manager in the game. In this way, we can make sure both elements are centered vertically and placed on opposite sides of the screen.

```
create() {
  ... // Other code
  // Create player
  this.player = this.add.image(40, this.scale.height / 2, 'player');
  this.player.setScale(0.5);

  // Create goal
  this.goal = this.add.image(this.scale.width - 80, this.scale.height / 2, 'treasure'
);
  this.goal.setScale(0.6);
}
```

Setting Up Collision Detection

To set up collision detection for our game, we will use some of Phaser's built-in utility methods that allow us to check for overlaps between two game objects using geometrical boundaries. No physics engine is needed for this basic detection.

In our update method, we will create two rectangles for our player and treasure chest game objects and check if they intersect:

```
update() {
  if (this.input.activePointer.isDown) {
    this.player.x += this.playerSpeed;
  }

  const playerRect = this.player.getBounds();
  const goalRect = this.goal.getBounds();

  if (Phaser.Geom.Intersects.RectangleToRectangle(playerRect, goalRect)) {
    console.log('reached goal');
    this.scene.restart();
  }
}
```

In this code:

- getBounds returns a rectangle shape that has the position, width, and height of our game object.
- Phaser.Geom.Intersects.RectangleToRectangle checks if two rectangles intersect. If they do, we reload the scene from scratch.

Recap

In this lesson, we:

1. Set up our player to move across the screen with input detection.
2. Created a new game object for our treasure chest and added it as our goal for the game.
3. Implemented basic collision detection by checking to see when our two game objects overlap.
4. Used the overlap as a trigger to restart our scene.

Awesome work! You now have a moving player and a goal to reach. You’re starting to see how Phaser handles real game interactions, and it’s only going to get more fun from here. Make sure to play around with it, tweak the speed, and experiment. We’ll see you in the next lesson, where we’ll add even more obstacles and mechanics. Happy coding!

In this lesson, we will add an exciting new feature to our game: a bouncing dragon enemy. This enemy will move up and down between two boundaries, adding a bit of unpredictability to keep the game engaging. Let's dive into the steps to achieve this.

Adding the Dragon Enemy

First, we need to add the dragon enemy to our scene. Since we have already loaded the dragon image earlier, we can now create a game object for it and set its scale.

Let's start by adding the dragon game object to our scene. We will do this at the end of our create method, after creating the goal.

```
this.enemy = this.add.image(120, this.scale.height / 2, 'enemy');
```

Next, we need to flip the dragon game object to face the player and set its scale:

```
this.enemy.flipX = true;
this.enemy.setScale(0.6);
```

Moving the Dragon Enemy

To make the dragon move up and down, we need to update its y value in the update method. We will start by adding a return statement in the if statement where the player reaches the goal. This ensures that the rest of the code in the update method does not run once the player reaches the goal.

```
    if (Phaser.Geom.Intersects.RectangleToRectangle(playerRect, goalRect)) {
      ...

      // make sure we leave this method
      return;
    }
```

Outside this if statement, we will update the enemy's y value by some speed. Initially, we will hardcode a value of 1:

```
this.enemy.y += 1;
```

To make the dragon move upwards, we subtract 1 from its y value:

```
this.enemy.y -= 1;
```

Instead of hardcoding the speed value, we will add it as a property on the enemy game object. At the bottom of the create method, add the following line:


```
this.enemy.speed = 1;
```

Now, update the code in the update method to use this new property:

```
this.enemy.y += this.enemy.speed;
```

Reversing the Dragon's Direction

To make the dragon bounce between two points, we need to reverse its direction once it reaches a certain y value. We will add two new variables for the conditions that we need to check:

```
const conditionUp = this.enemy.speed < 0 && this.enemy.y  0 <= 80;
const conditionDown = this.enemy.speed > 0 && this.enemy.y >= 280;
```

If either of these conditions is true, we will reverse the dragon's direction by multiplying its speed value by -1:

```
if (conditionUp || conditionDown) {
  this.enemy.speed *= -1;
}
```

Adding Randomness

To make the game more interesting, we will give the dragon a random speed and direction each time the scene starts. In the create method, after creating the enemy game object, we will choose a random direction:

```
const dir = Math.random() < 0.5 ? 1 : -1;
```

Next, we will add properties for the enemy's minimum and maximum speed in the init method:

```
this.enemyMinSpeed = 1.5;
this.enemyMaxSpeed = 3;
```

To create a random speed, we will use the Phaser.Math.RND.between method. We can add this after we create our random direction per the above:

```
const speed = Phaser.Math.RND.between(this.enemyMinSpeed, this.enemyMaxSpeed);
```

Finally, update the speed property on the enemy game object:

```
this.enemy.speed = dir * speed;
```

Setting Boundaries

Instead of hardcoding the boundary values in the update method, we will add them as properties in the init method:

```
this.enemyMinY = 80;
this.enemyMaxY = 280;
```

Now, update the conditions in the update method to reference these new properties:

```
const conditionUp = this.enemy.speed < 0 && this.enemy.y <= this.enemyMinY;
const conditionDown = this.enemy.speed > 0 && this.enemy.y >= this.enemyMaxY;
```

Recap

In this lesson, we covered the following steps to add a bouncing dragon enemy to our game:

- Added the dragon enemy game object and set its scale.
- Updated the update method to move the dragon up and down.
- Set boundaries to make the dragon bounce between two points.
- Added random speed and direction to keep the gameplay fresh.

Great job! You now have a bouncing enemy in your game, moving differently each time you play. In the next lesson, we will build on this by adding multiple enemies. Until then, keep experimenting and practicing!

In this lesson, we will enhance our enemy logic by introducing a useful Phaser concept: groups. A group is a collection of game objects that can be managed together, making it perfect for situations where we want to create multiple enemies that behave similarly.

Creating a Phaser Group

To begin, we will create a new Phaser group and add a few enemy game objects to it. Instead of creating each enemy manually, we will use the group system to spawn multiple game objects at once, evenly spaced apart.

Let's start by creating our group game object in the create method. We will store a reference to this group on our class:

```
this.enemies = this.add.group();
```

This method creates a new group game object. For now, let's log this new game object to understand its structure:

```
console.log(this.enemies);
```

In your browser's console, you will see that the group game object is an array of child game objects. It has utility functions for accessing and modifying these game objects.

Adding Game Objects to the Group

We can add game objects to the group in two ways:

- Dynamically create game objects when instantiating the group.
- Add other game objects to the group after they are created.

For example, we can add our previously created enemy game object to the group:

```
this.enemies.add(this.enemy);
```

Now, the group has one child, which is our enemy game object.

Dynamically Creating Game Objects

Instead of manually creating game objects, we can dynamically create them when we create our group game object. We do this by passing an additional argument to the this.add.group() method:

```
this.enemies = this.add.group({
  key: 'enemy',
  repeat: 4,
  setXY: {
    x: 90,
    y: 100,
    stepX: 100,
    stepY: 20
```



```
  }
});
```

Here’s what each property does:

- key: Specifies the asset key for the image game objects.
- repeat: Specifies how many additional game objects to create.
- setXY: Configures the starting x and y values and the spacing between game objects.

With this configuration, we create five enemy game objects, evenly spaced apart.

Scaling and Flipping Enemies

To scale and flip our enemies, we will use Phaser’s built-in action system. This system allows us to run a function across every member of the group in one step.

To scale our game objects, we use the Phaser.Actions.ScaleXY method:

```
Phaser.Actions.ScaleXY(this.enemies.getChildren(), -0.4, -0.4);
```

To flip our game objects, we use the Phaser.Actions.Call method:

```
Phaser.Actions.Call(this.enemies.getChildren(), (enemy) => {
  enemy.flipX = true;
});
```

These actions save us from writing repetitive loops and ensure all enemies share the same visual setup.

Adding Unique Behavior to Enemies

Finally, let’s add a random vertical speed and direction to each enemy game object. We will copy the block of code where we created the random direction and speed variables and assign them to each enemy in the group:

```
Phaser.Actions.Call(this.enemies.getChildren(), (enemy) => {
  enemy.flipX = true;

  // Set enemy speed
  const dir = Math.random() < 0.5 ? 1 : -1;
  const speed = Phaser.Math.RND.between(this.enemyMinSpeed, this.enemyMaxSpeed);
  enemy.speed = dir * speed;
});
```

With these changes, our create function will now look like this:

```
// called once after the preload ends
```



```
create() {
  // create bg image
  const bg = this.add.image(0, 0, 'background');

  // change origin to the top-left corner
  bg.setOrigin(0, 0);

  // create the player
  this.player = this.add.image(40, this.scale.height / 2, 'player');

  // we are reducing the width and height by 50%
  this.player.setScale(0.5);

  // goal
  this.goal = this.add.image(this.scale.width - 80, this.scale.height / 2, 'treasure'
);
  this.goal.setScale(0.6);

  this.enemies = this.add.group({
    key: 'enemy',
    repeat: 4,
    setXY: {
      x: 90,
      y: 100,
      stepX: 100,
      stepY: 20
    }
  });
  Phaser.Actions.ScaleXY(this.enemies.getChildren(), -0.4, -0.4);
  Phaser.Actions.Call(this.enemies.getChildren(), (enemy) => {
    enemy.flipX = true;

    // set enemy speed
    const dir = Math.random() < 0.5 ? 1 : -1;
    const speed = Phaser.Math.RND.between(this.enemyMinSpeed, this.enemyMaxSpeed);
    enemy.speed = dir * speed;
  });
}
```

This concludes our lesson on adding and managing enemy groups in Phaser. In the next lesson, we will continue working on our enemy group to enhance their behavior further.

In this lesson, we will continue working on adding our enemy group to our game. Our goal is to make all the dragons move and bounce between two vertical limits. We will also implement collision detection between the player and the enemies. By the end of this lesson, our game will have real stakes, making it more challenging and engaging for the players.

Updating Enemy Movement

To make all our dragons move, we need to update our code in the update function to loop through each of our enemies and make them bounce between our two vertical limits. We will reuse the same logic we used previously but apply it to every group member.

First, let's grab all of our enemy game objects from our group. We do this by using the getChildren method, which returns an array of game objects.

```
// this is called up to 60 times per second
update() {
  // check for active input (left click / touch)
  ...

  // treasure overlap check
  ...
  }

  const enemies = this.enemies.getChildren();
}
```

Next, we need to loop through our array of game objects and update each one individually. We'll use a for loop for this purpose.

```
for (let i = 0; i < enemies.length; i += 1) {
  // Enemy movement logic will go here
}
```

Inside the for loop, we will add the logic to make each enemy bounce between the vertical limits. We need to update the enemy's position and check if it has reached the limits to reverse its direction.

```
    for (let i = 0; i < enemies.length; i += 1) {
      // enemy movement
      enemies[i].y += enemies[i].speed;

      // check we haven't passed min or max Y
      const conditionUp = enemies[i].speed < 0 && enemies[i].y <= this.enemyMinY;
      const conditionDown = enemies[i].speed > 0 && enemies[i].y >= this.enemyMaxY;

      // if we passed the upper or lower limit, reverse
      if (conditionUp || conditionDown) {
        enemies[i].speed *= -1;
      }
}
```


Removing Original Enemy Code

Now that we have all our dragons in the group moving, we can remove the code tied to our original dragon enemy (if you haven't done so already). This includes the code for creating the enemy and setting its speed in the create method.

Implementing Collision Detection

Finally, we will add a collision check between our player and each of our enemy game objects. If our player overlaps with an enemy, we will log a message and restart the scene.

To do this, we need to create a rectangle for our enemy game object using the getBounds method. Then, we can check if our player rectangle overlaps with the enemy rectangle. We will copy the overlap check code from the player and goal overlap check and modify it for the player and enemy overlap check.

```
const enemyRect = enemies[i].getBounds();
if (Phaser.Geom.Intersects.RectangleToRectangle(playerRect, enemyRect)) {
  console.log('Game over!');
  this.scene.restart();
  return;
}
```

Now, when the player overlaps with an enemy, we log the message "Game over!" and handle the game over scenario.

Recap

In this lesson, we accomplished the following:

- Created a new Phaser group of our enemy sprites.
- Positioned the game objects using setXY when we created the Phaser group.
- Used Phaser actions to scale and flip the game objects.
- Gave each of our enemies a random speed and direction.
- Refactored our bouncing movement logic to work with all the enemies in our group.
- Implemented collision detection between our player and the enemies in our group.

Challenge

Try updating the number of spawned enemies and the space between each of the spawned enemies. You can experiment with the starting positions to get a wider range of changes for your game.

In the next lesson, we will go over the solution before moving on to adding polishing effects for our game.

In the previous video, we challenged you to update the total number of spawned enemies to 6 and adjust their spacing. In this lesson, we will walk through a possible solution to this challenge. Let's dive into the steps required to make these changes.

Updating the Number of Enemies

To modify the number of enemies in our game, we need to adjust the properties of the Phaser group that manages our enemy game objects. Here's how you can do it:

1. Navigate to the create method in your game-scene.js file.
2. Locate the section where the Phaser group for enemies is created.
3. Update the repeat property to 5, which will result in a total of 6 enemy game objects (since the initial object is also counted).

Here is the relevant code snippet:

```
this.enemies = this.add.group({
  key: 'enemy',
  repeat: 5,
  setXY: {
    x: 90,
    y: 100,
    stepX: 100,
    stepY: 20
  }
});
```

Adjusting the Spacing Between Enemies

After updating the number of enemies, you might notice that the last dragon appears past the treasure chest goal. To fix this, we need to adjust the spacing between the enemies. This can be done by modifying the stepX property in the setXY object.

1. In the same section where you updated the repeat property, locate the stepX property.
2. Change the value of stepX to adjust the horizontal spacing between the enemies. For example, setting it to 80 will bring the dragons closer together.

Here is the updated code snippet with the new spacing:

```
this.enemies = this.add.group({
  key: 'enemy',
  repeat: 5,
  setXY: {
    x: 90,
    y: 100,
    stepX: 80,
    stepY: 20
  }
});
```

However, a spacing of 80 might be too tight for gameplay. You can further adjust this value to find

the optimal spacing. For instance, setting stepX to 90 might provide a better layout:

```
this.enemies = this.add.group({
  key: 'enemy',
  repeat: 5,
  setXY: {
    x: 90,
    y: 100,
    stepX: 90,
    stepY: 20
  }
});
```

Final Adjustments

After making these changes, observe the positioning of the enemies. The goal is to ensure that the last dragon appears to be guarding the treasure chest, enhancing the gameplay experience. You might need to tweak the stepX value slightly to achieve the perfect layout.

By following these steps, you should be able to update the number of enemies and adjust their spacing effectively. This will make your game more challenging and visually appealing.

Note for the next lesson, we'll return to our original setup. However, feel free to play around with the values until you find something that you like!

Welcome back! So far, our game is looking great, but in this lesson, we're going to make it feel even more alive with camera effects and event listening. We'll shake the screen, fade it out, and learn how to respond to events in Phaser to make those transitions feel smooth and polished.

Refactoring the Game Over Logic

Currently, when our player collides with an enemy or reaches the goal, the scene restarts immediately. This works, but it's a bit abrupt. To enhance this, we'll update our game to fade out once the game is over. We'll start by moving our restart logic into its own method, which we'll call handleGameOver.

In our game scene, let's add a new method at the bottom of our class:

```
handleGameOver() {
  this.scene.restart();
}
```

Now, we need to update our logic in the update method to call handleGameOver instead of this.scene.restart.

Find the line where we restart the scene when the player reaches the goal:

```
if (Phaser.Geom.Intersects.RectangleToRectangle(playerRect, goalRect)) {
  console.log('reached goal!!');
  this.handleGameOver();
  return;
}
```

And the line where we restart the scene when the player collides with an enemy:

```
if (Phaser.Geom.Intersects.RectangleToRectangle(playerRect, enemyRect)) {
  console.log('Game over!');
  this.handleGameOver();
  return;
}
```

Adding a Camera Shake Effect

To make the moment more dramatic when our player dies, we'll use Phaser's built-in camera system to apply a quick shake effect. Here's how to do it:

```
handleGameOver() {
  this.cameras.main.shake(500);
  this.scene.restart();
}
```

To test the shake effect, temporarily comment out the line where we restart the scene:


```
handleGameOver() {
  this.cameras.main.shake(500);
  // this.scene.restart();
}
```

Now, when you move your player and collide with an enemy, you should see the shake effect. This small shake instantly makes the game feel more responsive and reactive, adding a burst of feedback when something big happens.

Waiting for the Shake to Finish

Currently, the shake plays and then the scene immediately restarts. We want to wait until the shake is finished before restarting the scene. We'll set this up with an event listener.

Phaser emits an event when the shake effect is finished. We can listen for this event and then run our restart code. Here's how to do it:

```
handleGameOver() {
  this.cameras.main.shake(500);
  this.cameras.main.once(Phaser.Cameras.Scene2D.Events.SHAKE_COMPLETE, () => {
    this.scene.restart();
  });
}
```

Preventing Multiple Game Over Calls

You might notice that the game over message is being logged multiple times when the player collides with an enemy. This happens because our handleGameOver method is being called multiple times in the update loop. To fix this, we'll add a simple flag to track whether the game is already ending.

In the init method, add a new property to our class:

```
init() {
  this.isGameOver = false;
}
```

In the handleGameOver method, set this property to true:

```
handleGameOver() {
  this.isGameOver = true;
  this.cameras.main.shake(500);
  this.cameras.main.once(Phaser.Cameras.Scene2D.Events.SHAKE_COMPLETE, () => {
    this.scene.restart();
  });
}
```


In the update method, check this value and return early if it’s true:

```
update() {
  if (this.isGameOver) {
    return;
  }
  // rest of the update code
}
```

Conditional Camera Shake

We only want to shake the camera if the player dies from colliding with an enemy, not when the player reaches the goal. To make this change, we’ll update our handleGameOver method to expect an argument that indicates whether the player died.

Update the handleGameOver method to accept a parameter, and use that parameter to only run our shake if the player died:

```
handleGameOver(playerDied) {
  this.isGameOver = true;
  if (playerDied) {
    this.cameras.main.shake(500);
    this.cameras.main.once(Phaser.Cameras.Scene2D.Events.SHAKE_COMPLETE, () => {
      this.scene.restart();
    });
  } else {
    this.scene.restart();
  }
}
```

Update the calls to handleGameOver in the update method to pass the appropriate argument:

For collision with enemies:

```
this.handleGameOver(true);
```

For reaching the goal:

```
this.handleGameOver(false);
```

Adding a Fade Out Effect

Finally, we’ll chain one more effect for when our game is over: a nice fade-out effect where the screen will fade to black. We only want to trigger this after the screen shake ends. So, instead of triggering the scene restart, we’ll fade the camera first. We can then check when the fade out completes to restart our scene (another event trigger Phaser receives callbacks about).

Update the handleGameOver method to include the fade-out effect:


```
handleGameOver(playerDied) {
  this.isGameOver = true;
  if (playerDied) {
    this.cameras.main.shake(500);
    this.cameras.main.once(Phaser.Cameras.Scene2D.Events.SHAKE_COMPLETE, () => {
      this.cameras.main.fadeOut(500);
    });
  } else {
    this.cameras.main.fadeOut(500);
  }

  this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE, () => {
    this.scene.restart();
  });
}
```

Recap

Here’s a quick recap of what we accomplished in this lesson:

- Refactored our scene restart into a reusable handleGameOver method.
- Used camera effects like shake and fade out.
- Responded to these camera events with event listeners.
- Used a flag to prevent unintended repeated behavior.

Amazing work! You just added polish and personality to your game using simple but powerful techniques. In our next lesson, we’ll explore how to add a title screen to tie all of our scenes together. See you there!

In this lesson, we will enhance our game by adding a title screen. A well-designed title screen sets the tone for your game, giving players a moment to prepare before the action begins. We'll implement this as a new Phaser Scene, which helps keep our project organized and our code maintainable. Let's dive in!

Creating a New Phaser Scene

To create a new Phaser scene, follow these steps:

1. In your project's scenes folder, create a new file named title-scene.js.
2. Copy the code from your existing game-scene.js file and paste it into title-scene.js.

Modifying the Title Scene

Next, we need to modify the title-scene.js file to suit our title screen requirements. Here's how you can do it:

1. Remove the logic for handling game over and the update method.
2. In the create method, remove all the code but keep the method definition.
3. In the preload method, remove all the code except for the background image loading code.
4. Remove the init method.
5. Update the class name to TitleScene and ensure the scene key matches the class name.

Your title-scene.js file should now look like this:

```
// create a new scene
export class TitleScene extends Phaser.Scene {
  constructor() {
    super({
      key: 'TitleScene'
    });
  }

  // load assets
  preload() {
    // load images
    this.load.image('background', 'assets/background.png');
  }

  // called once after the preload ends
  create() {
  }
}
```

Updating the Game Configuration

Now, we need to update our game configuration to include the new title scene. Open your main.js file and follow these steps:

1. Copy the import statement for GameScene and paste it below, updating it to import TitleScene from title-scene.js.
2. Update the game configuration to include an array of scenes, with TitleScene as the first

element.

Your main.js file should look like this:

```
import { GameScene } from './scenes/game-scene.js';
import { TitleScene } from './scenes/title-scene.js';

// set the configuration of the game
const config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  scene: [TitleScene, GameScene],
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
  backgroundColor: '#000000'
};

// create a new game, pass the configuration
const game = new Phaser.Game(config);
```

Adding Game Objects to the Title Scene

With the new scene in place, let’s add some game objects to our title screen. We’ll start by adding a background image and some text.

Adding a Background Image

Update the preload method to load the title background image:

```
preload() {
  // load images
  this.load.image('titleBackground', 'assets/title-background.png');
}
```

Next, update the create method to display the background image:

```
create() {
  const bg = this.add.image(0, 0, 'titleBackground');
  bg.setOrigin(0, 0);
}
```

Adding Text to the Title Scene

To add text to the title screen, we’ll create text game objects. These objects allow us to display text and apply custom styling.

Add the following code to the create method to display the game title and a prompt to start the game:

```
create() {
  const bg = this.add.image(0, 0, 'titleBackground');
  bg.setOrigin(0, 0);

  this.add.text(this.scale.width / 2, this.scale.height / 2 - 50, 'Crossing Road Adve
nture', {
    fontSize: '28px'
  }).setOrigin(0.5);

  this.add.text(this.scale.width / 2, this.scale.height / 2 + 50, 'Click to play!', {
    fontSize: '18px'
  }).setOrigin(0.5);
}
```

Adding Camera Fade Effects

To make the title screen appear more cinematic, we'll add a fade-in effect when the scene starts. Update the create method as follows:

```
create() {
  const bg = this.add.image(0, 0, 'titleBackground');
  bg.setOrigin(0, 0);

  this.add.text(this.scale.width / 2, this.scale.height / 2 - 50, 'Crossing Road Adve
nture', {
    fontSize: '28px'
  }).setOrigin(0.5);

  this.add.text(this.scale.width / 2, this.scale.height / 2 + 50, 'Click to play!', {
    fontSize: '18px'
  }).setOrigin(0.5);

  this.cameras.main.fadeIn(500);
}
```

Transitioning to the Game Scene

Finally, we'll make the game start when the player clicks or taps on the screen. We'll use Phaser's input system to detect the interaction and transition to the game scene with a fade-out effect.

Add the following code to the create method to handle the click event:

```
create() {
  const bg = this.add.image(0, 0, 'titleBackground');
  bg.setOrigin(0, 0);

  this.add.text(this.scale.width / 2, this.scale.height / 2 - 50, 'Crossing Road Adve
```



```
nture', {
    fontSize: '28px'
  }).setOrigin(0.5);

  this.add.text(this.scale.width / 2, this.scale.height / 2 + 50, 'Click to play!', {
    fontSize: '18px'
  }).setOrigin(0.5);

  this.cameras.main.fadeIn(500);

  this.input.once('pointerup', () => {
    this.cameras.main.fadeOut(500);

    this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE, () => {
      this.scene.start('GameScene');
    });
  });
}
```

Since we're fading out the camera, we need to fade it in when the Game Scene is loaded. In the create method of **game-scene.js**, add the following line so the camera fades in when the game starts:

```
this.cameras.main.fadeIn(500);
```

That's it! With these steps, you've created a polished title screen that enhances the player's experience. In the next lesson, we'll add a simple score and level system to track the player's progress. See you there!

In this lesson, we will add an essential feature to our game: a score system. This system will track the number of levels the player completes. Each time the player reaches the goal, the scene will restart, but the score will continue to increase and be displayed on the screen. Let's get started.

Setting Up the Score Tracker

To keep track of the levels completed by the player, we need to create a variable. Since Phaser's init method runs before each scene starts, it is an ideal place to initialize this variable.

Open your game-scene.js file and locate the init method. Add the following code to create a new property called levelsCompleted and set it to zero:

```
init() {
  this.levelsCompleted = 0;

  // Other init variables
}
```

Displaying the Score

Next, we need to display the score on the game scene. We will create a text game object for this purpose. In the create method, after the fade-in effect, add the following code to create the text game object:

```
this.add.text(10, 10, `Score: ${this.levelsCompleted * 100}`, { fontSize: '22px' });
```

This code creates a text object at position (10, 10) with the string "Score:" followed by the levels completed multiplied by 100. The font size is set to 22 pixels.

Updating the Score

To increase the score when the player reaches the goal, we need to update the handleGameOver method. Before restarting the Phaser scene, we will check if the player died. If the player died, we reset the level counter to zero. Otherwise, we increment the level counter by one.

Locate the handleGameOver method and add the following check after the fadeout completes:

```
handleGameOver(playerDied) {
  this.isGameOver = true;

  if (playerDied) {
    this.cameras.main.shake(500);
    this.cameras.main.once(Phaser.Cameras.Scene2D.Events.SHAKE_COMPLETE, () => {
      this.cameras.main.fadeOut(500);
    });
  } else {
    this.cameras.main.fadeOut(500);
  }

  this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE, () => {
    if (playerDied) {
```



```
      this.levelsCompleted = 0;
    } else {
      this.levelsCompleted += 1;
    }

    // restart the Scene
    this.scene.restart();
  });
}
```

Preserving the Score on Scene Restart

To ensure that the score is preserved when the scene restarts, we need to update the init method. We will add a check to see if levelsCompleted is undefined. If it is, we initialize it to zero.

Update the init method as follows:

```
init() {
  if (this.levelsCompleted === undefined) {
    this.levelsCompleted = 0;

  // Other init variables
  }
}
```

Testing the Changes

To make it easier to test our changes, we will temporarily move the game scene in front of the title screen in the main.js file. Open main.js and move the game scene to the first element in the scene array:

```
const config = {
  type: Phaser.AUTO,
  scene: [GameScene, TitleScene],
  // other configurations...
};
```

Save the file and refresh the game. You should see the new text game object for the score appear in the level. Now, when you move the player across the screen and reach the goal, the score should increase. If the player touches an enemy, the scene should restart, and the score should go back to zero.

Finally, don’t forget to update the game configuration to start with the title scene before the game scene. In the main.js file, move the title scene to be the first element in the scene array:

```
const config = {
  type: Phaser.AUTO,
  scene: [TitleScene, GameScene],
  // other configurations...
};
```


Recap

In this lesson, we added the following features to our game:

- A level counter that increments with each completed goal.
- A UI text element that displays the current score.
- Preservation of the score on scene restarts, making the game feel more progressive.

Great work! You have now implemented a score system that provides a sense of progression and purpose to the game. In the next video, we will do a quick course wrap-up. See you there!

Congratulations on completing your Phaser 4 course! Your instructor, Scott Westover, has guided you through a comprehensive journey into game development using this powerful HTML5 game engine. Throughout this course, you've gained valuable skills and created an engaging 2D browser game. Let's recap what you've learned and explore why Phaser 4 is an excellent tool for your projects.

Why Phaser 4?

- **Cross-platform compatibility:** Your game runs seamlessly in the browser without any installation requirements.
- **Open-source and free:** Phaser has a vibrant community, and there are no licensing fees, making it an accessible choice for developers.
- **Perfect for 2D games:** From platformers to puzzles, Phaser makes it easy to create a variety of 2D games.
- **Modern JavaScript:** Phaser is written with clean, efficient, and developer-friendly modern JavaScript.
- **Rich feature set:** Phaser covers a wide range of features, including animation, physics, input, and audio.

Your Game Project

Throughout this course, you built a Frogger-style game where the player guides a character across a dragon-filled road to reach a treasure chest. While the game may look simple, you implemented powerful mechanics such as:

- Player movement
- Collision detection
- Animated transitions
- Level tracking system

By the end of the course, you created a fully playable game that you can continue to expand, refine, and showcase as a portfolio project.

Course Summary

Here's a quick recap of the topics covered throughout the course:

1. Setting up a Phaser project and understanding the structure of a Phaser scene
2. Creating and manipulating game objects, including images, text, and sprites
3. Handling user input to control the player's movement
4. Detecting collisions between the player and other game objects
5. Applying camera effects like shake and fade for visual polish
6. Controlling win and loss conditions and providing gameplay feedback
7. Using scenes and transitions to structure the game cleanly
8. Implementing a level and score system to track player progress

Your Learning Journey with Zenva

Zenva is an online learning academy with over a million students, offering a wide range of courses for beginners and those looking to learn new skills. Our courses are versatile, allowing you to learn in various ways:

- Watch video tutorials
- Read lesson summaries
- Follow along with the instructor using included project files

Next Steps

We hope you enjoyed building your game and gained confidence in creating more Phaser projects independently. Remember, there’s always more to explore! Keep experimenting and learning. Until next time, happy coding!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Scaling and Flipping Practice

index.html

Found in folder: **/Practice – Scaling and Flipping**

This code sets up an HTML file for a game development project. It includes the Phaser library and a main JavaScript file. The HTML file also contains some basic styling to remove padding and margins from the page.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Learn Game Development at Zenva.com</title>
    <style>
      html, body {
        padding: 0px;
        margin: 0px;
      }
    </style>
  </head>
  <body>
    <script src="src/lib/phaser.js"></script>
    <script src="src/main.js" type="module"></script>
  </body>
</html>
```

main.js

Found in folder: **/Practice – Scaling and Flipping/src**

This code imports the GameScene class from the ‘./scenes/game-scene.js’ file. It then sets up the configuration for a Phaser game, specifying the game scene, dimensions, scaling mode, and background color. Finally, it creates a new Phaser game instance using the provided configuration.

```
import { GameScene } from './scenes/game-scene.js';

// set the configuration of the game
const config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  scene: GameScene,
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
```



```
  backgroundColor: '#000000',
};

// create a new game, pass the configuration
const game = new Phaser.Game(config);
```

game-scene.js

Found in folder: **/Practice – Scaling and Flipping/src/scenes**

This code defines a new Phaser scene called “GameScene”. The scene loads assets such as images for the background, player, and enemies in the preload method. In the create method, it sets up the game environment by adding and positioning the loaded images on the screen, adjusting their properties like scale, origin, and depth.

```
// create a new scene
export class GameScene extends Phaser.Scene {
  constructor() {
    super({
      key: 'GameScene',
    });
  }

  // initialize game properties
  init() {
    console.log('init method called');
  }

  // load assets
  preload() {
    console.log('preload method called');
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
  }

  // called once after the preload ends
  create() {
    console.log('create method called');
    const gameW = this.scale.width;
    const gameH = this.scale.height;
    console.log(gameW, gameH);

    const player = this.add.image(0, 0, 'player');
    player.setPosition(gameW, gameH);
    player.setOrigin(1, 1);
    player.setDepth(2);
    player.setScale(0.5);

    // create bg image
    const bg = this.add.image(0, 0, 'background');
```



```
    // place image in the center of the screen
    bg.setPosition(gameW / 2, gameH / 2);

    // change origin to the top-left corner
    // bg.setOrigin(0, 0);

    const enemy1 = this.add.image(250, 180, 'enemy');
    enemy1.scaleX = 2;
    enemy1.scaleY = 2;

    const enemy2 = this.add.image(450, 180, 'enemy');
    enemy2.displayWidth = 300;

    enemy1.flipX = true;
    enemy1.flipY = true;
  }

  // called for each step of our game loop
  update() {
    // console.log('update method called');
  }
}
```

Rotation and Updating Practice

index.html

Found in folder: **/Practice – Rotation and Updating**

This code sets up an HTML file for a game development project. It includes the Phaser library and a main JavaScript file. The HTML file also contains some basic styling to remove padding and margins from the page.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Learn Game Development at Zenva.com</title>
    <style>
      html, body {
        padding: 0px;
        margin: 0px;
      }
    </style>
  </head>
  <body>
    <script src="src/lib/phaser.js"></script>
    <script src="src/main.js" type="module"></script>
  </body>
</html>
```

main.js

Found in folder: **/Practice – Rotation and Updating/src**

This code imports the GameScene class from the ‘./scenes/game-scene.js’ file. It then sets up the configuration for a Phaser game, specifying the game type, scene, scale, and background color. Finally, it creates a new Phaser game instance using the defined configuration.

```
import { GameScene } from './scenes/game-scene.js';

// set the configuration of the game
const config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  scene: GameScene,
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
  backgroundColor: '#000000',
};

// create a new game, pass the configuration
const game = new Phaser.Game(config);
```

game-scene.js

Found in folder: **/Practice – Rotation and Updating/src/scenes**

This code defines a new Phaser scene called “GameScene”. In the preload method, it loads the necessary game assets such as background, player, and enemy images. The create method sets up the initial game state by adding the background, player, and enemy to the scene, while the update method is called in each game loop iteration, handling the rotation and scaling of the player and enemy.

```
// create a new scene
export class GameScene extends Phaser.Scene {
  constructor() {
    super({
      key: 'GameScene',
    });
  }

  // load assets
  preload() {
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
  }

  // called once after the preload ends
  create() {
    // create bg image
    const bg = this.add.image(0, 0, 'background');
```



```
    // change origin to the top-left corner
    bg.setOrigin(0, 0);

    this.player = this.add.image(70, 180, 'player');
    this.player.setScale(0.5);

    this.enemy1 = this.add.image(250, 180, 'enemy');
    // enemy1.setAngle(45);
    // enemy1.rotation = Math.PI / 4;
    // enemy1.setOrigin(0);
    this.enemy1.setRotation(Math.PI / 4);
  }

  // called for each step of our game loop
  update() {
    // console.log('update method called');
    this.enemy1.angle += 1;

    this.player.angle -= 1;
    if (this.player.scaleX < 2) {
      this.player.scaleX += 0.01;
      this.player.scaleY += 0.01;
    }
  }
}
```

Final Game Project

index.html

Found in folder: **/Game Project**

This code sets up an HTML file for a game development project. It includes the Phaser library and a main JavaScript file. The HTML file also contains some basic styling to remove padding and margins from the page.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Learn Game Development at Zenva.com</title>
    <style>
      html, body {
        padding: 0px;
        margin: 0px;
      }
    </style>
  </head>
  <body>
    <script src="src/lib/phaser.js"></script>
    <script src="src/main.js" type="module"></script>
  </body>
```



```
</html>
```

main.js

Found in folder: **/Game Project/src**

This code imports two scenes, GameScene and TitleScene, from their respective files. It then sets up the configuration for a Phaser game, specifying the game type, scenes, scale, and background color. Finally, it creates a new Phaser game instance using the defined configuration.

```
import { GameScene } from './scenes/game-scene.js';
import { TitleScene } from './scenes/title-scene.js';

// set the configuration of the game
const config = {
  type: Phaser.AUTO, // Phaser will use WebGL if available, if not it will use Canvas
  scene: [TitleScene, GameScene],
  scale: {
    width: 640,
    height: 360,
    mode: Phaser.Scale.FIT,
  },
  backgroundColor: '#000000',
};

// create a new game, pass the configuration
const game = new Phaser.Game(config);
```

game-scene.js

Found in folder: **/Game Project/src/scenes**

This code defines a Phaser game scene called GameScene. The scene includes a player, enemies, and a treasure goal. The player moves towards the goal while avoiding enemies, and the round ends when the player reaches the goal or collides with an enemy.

```
// create a new scene
export class GameScene extends Phaser.Scene {
  constructor() {
    super({
      key: 'GameScene',
    });
  }

  init() {
    if (this.levelsCompleted === undefined) {
      this.levelsCompleted = 0;
    }

    // player speed
    this.playerSpeed = 3;
```



```
    // enemy speed
    this.enemyMinSpeed = 1.5;
    this.enemyMaxSpeed = 3;

    // boundaries
    this.enemyMinY = 80;
    this.enemyMaxY = 280

    this.isGameOver = false;
  }

  // load assets
  preload() {
    // load images
    this.load.image('background', 'assets/background.png');
    this.load.image('player', 'assets/player.png');
    this.load.image('enemy', 'assets/dragon.png');
    this.load.image('treasure', 'assets/treasure.png');
  }

  // called once after the preload ends
  create() {
    // create bg image
    const bg = this.add.image(0, 0, 'background');

    // change origin to the top-left corner
    bg.setOrigin(0, 0);

    // create the player
    this.player = this.add.image(40, this.scale.height / 2, 'player');

    // we are reducing the width and height by 50%
    this.player.setScale(0.5);

    // goal
    this.goal = this.add.image(this.scale.width - 80, this.scale.height / 2, 'treasur
e');
    this.goal.setScale(0.6);

    this.enemies = this.add.group({
      key: 'enemy',
      repeat: 4,
      setXY: {
        x: 90,
        y: 100,
        stepX: 100,
        stepY: 20
      }
    });
    Phaser.Actions.ScaleXY(this.enemies.getChildren(), -0.4, -0.4);
    Phaser.Actions.Call(this.enemies.getChildren(), (enemy) => {
      enemy.flipX = true;

      // set enemy speed
      const dir = Math.random() < 0.5 ? 1 : -1;
```

```
      const speed = Phaser.Math.RND.between(this.enemyMinSpeed, this.enemyMaxSpeed);
      enemy.speed = dir * speed;
    });

    this.cameras.main.fadeIn(500);

    this.add.text(10, 10, `Score: ${this.levelsCompleted * 100}`, { fontSize: '22px'
});
  }

  // this is called up to 60 times per second
  update() {
    if (this.isGameOver) {
      return;
    }

    // check for active input (left click / touch)
    if (this.input.activePointer.isDown) {
      // player walks
      this.player.x += this.playerSpeed;
    }

    // treasure overlap check
    const playerRect = this.player.getBounds();
    const goalRect = this.goal.getBounds();

    if (Phaser.Geom.Intersects.RectangleToRectangle(playerRect, goalRect)) {
      console.log('reached goal!!');

      this.handleGameOver(false);

      // make sure we leave this method
      return;
    }

    const enemies = this.enemies.getChildren();
    for (let i = 0; i < enemies.length; i += 1) {
      // enemy movement
      enemies[i].y += enemies[i].speed;

      // check we haven't passed min or max Y
      const conditionUp = enemies[i].speed < 0 && enemies[i].y <= this.enemyMinY;
      const conditionDown = enemies[i].speed > 0 && enemies[i].y >= this.enemyMaxY;

      // if we passed the upper or lower limit, reverse
      if (conditionUp || conditionDown) {
        enemies[i].speed *= -1;
      }

      const enemyRect = enemies[i].getBounds();
      if (Phaser.Geom.Intersects.RectangleToRectangle(playerRect, enemyRect)) {
        console.log('Game over!');

        this.handleGameOver(true);
```

```
        // make sure we leave this method
        return;
      }
    }
  }

  handleGameOver(playerDied) {
    this.isGameOver = true;

    if (playerDied) {
      this.cameras.main.shake(500);
      this.cameras.main.once(Phaser.Cameras.Scene2D.Events.SHAKE_COMPLETE, () => {
        this.cameras.main.fadeOut(500);
      });
    } else {
      this.cameras.main.fadeOut(500);
    }

    this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE, () => {
      if (playerDied) {
        this.levelsCompleted = 0;
      } else {
        this.levelsCompleted += 1;
      }
      // restart the Scene
      this.scene.restart();
    });
  }
}
```

title-scene.js

Found in folder: **/Game Project/src/scenes**

This code defines a new Phaser scene called “TitleScene”. In the preload function, it loads an image asset for the title background. The create function sets up the title screen by adding the background image, displaying the game title and a “Click to play!” message, and handling the click event to fade out the camera and start the “GameScene” once the fade-out is complete.

```
// create a new scene
export class TitleScene extends Phaser.Scene {
  constructor() {
    super({
      key: 'TitleScene',
    });
  }

  // load assets
  preload() {
    // load images
    this.load.image('titleBackground', 'assets/title-background.png');
  }
```



```
  // called once after the preload ends
  create() {
    const bg = this.add.image(0, 0, 'titleBackground');
    bg.setOrigin(0, 0);

    this.add.text(this.scale.width / 2, this.scale.height / 2 - 50, 'Crossing Road Adventure', {
      fontSize: '28px',
    })
      .setOrigin(0.5);

      this.add.text(this.scale.width / 2, this.scale.height / 2 + 50, 'Click to play!', {
      fontSize: '18px',
    })
      .setOrigin(0.5);

    this.cameras.main.fadeIn(500);

    this.input.once('pointerup', () => {
      this.cameras.main.fadeOut(500);
    });

    this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE, () => {
      this.scene.start('GameScene');
    });
  }
}
```